

UNIVERSIDAD DE MONTERREY
Escuela de Ingeniería y Tecnología
Ingeniería en Tecnologías Computacionales

Review de Seguridad

Autor(es)
600046 Fernando Olivares Del Valle

Profesor (a)
Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Review de Seguridad

1. Hash de contraseñas y política básica de contraseñas

- **Amenaza:** Si las contraseñas se guardan en texto plano y la base de datos fuera comprometida, todas las cuentas quedarían expuestas de inmediato.
- **Control aplicado:** Las contraseñas se hashean con bcrypt (factor de costo 12) antes de guardarse (authController.register, userController.save). El registro exige mínimo 8 caracteres (register.ejs, atributo minlength="8").
- **Evidencia:** En la tabla usuarios, la columna password_hash del admin de prueba contiene un valor con formato hash.
-

```
library-> SELECT * FROM usuarios WHERE nombre = 'Admin Principal'
library-> /
id_usuario | nombre | correo | password_hash | es_admin | fecha_registro | activo
-----|-----|-----|-----|-----|-----|-----
(1 row)
1 | Admin Principal | admin@libreria.com | $2b$12$GAjEFVah09aZVdcYlo9O2.4uXB5qg554tx0uD6kbBAn/kw7nBlpQu | t | 2026-08-30 02:47:34.566179 | t
library->
```

2. Variables de entorno para secretos y credenciales

- **Amenaza:** Publicar credenciales de base de datos o el secreto de sesión directamente en el código fuente permite a cualquiera con acceso al repositorio comprometer el sistema.
- **Control aplicado:** config/db.js y app.js leen DB_HOST, DB_USER, DB_PASSWORD, SESSION_SECRET, etc. desde .env mediante dotenv; solo .env.example (sin valores reales) se sube al repositorio.
- **Evidencia:** git ls-files no incluye .env en el listado de archivos rastreados del repositorio.

```
fodv004@maquina-01:/opt/udem/WebMonolitico
[fodv004@maquina-01 WebMonolitico]$ git ls-files | grep -i env
.env.example
[fodv004@maquina-01 WebMonolitico]$
```

3. Consultas SQL parametrizadas

- **Amenaza:** SQL Injection — un atacante inserta código SQL malicioso a través de un campo de formulario para leer, modificar o borrar datos sin autorización.
- **Control aplicado:** El 100% de las consultas en `src/models/` usa parámetros posicionales (`$1`, `$2...`) del driver pg; no existe concatenación de strings de usuario dentro de SQL en ninguna parte del proyecto.
- **Evidencia:** Se intentó iniciar sesión con el correo `admin@libreria.com` OR `'1'='1` — la consulta parametrizada trató el texto completo como un valor literal de búsqueda (no como código SQL), y el login fue rechazado con 401 en vez de otorgar acceso indebido.

```
fodv004@maquina-01:opt/udem/WebMonolitico
[fodv004@maquina-01 WebMonolitico]# curl -i -X POST http://127.0.0.1:3000/login \
-d "correo=admin@libreria.com" OR '1'='1' \
-d "password=cualquiera"
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 2189
ETag: W/"86d-RJdgVhdman2SHVnbMjfsB4Q2YE"
Date: Sun, 30 Aug 2026 15:53:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<!doctype html><html lang="es"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>Libreria</title><link rel="icon" href="data:image/svg+xml,
1,svg xmlns="http://www.w3.org/2000/svg"><svg width="4220" height="4220"><rect width="4220" height="4220" fill="white" stroke="black" stroke-width="1" /></svg></head><body><div class="auth-shell"><div class="auth-brand"><div class="logo"><img alt="Libreria logo" data-bbox="15 15 42 42" /></div><div class="text"><h1>Libreria</h1></div></div><div class="auth-form"><div class="form"><div class="input"><input type="text" value="correo" /></div><div class="input"><input type="password" value="password" /></div><div class="button"><button type="submit">Iniciar sesión</button></div></div></div><div class="auth-message"><div class="message"><div class="text"><h2>¡Bienvenido!</h2></div><div class="text"><p>¡Bienvenido a la libreria!</p></div></div></div><div class="auth-footer"><div class="text"><p>web monolitica MVC - PostgreSQL directo</p></div></div></body></html>
```

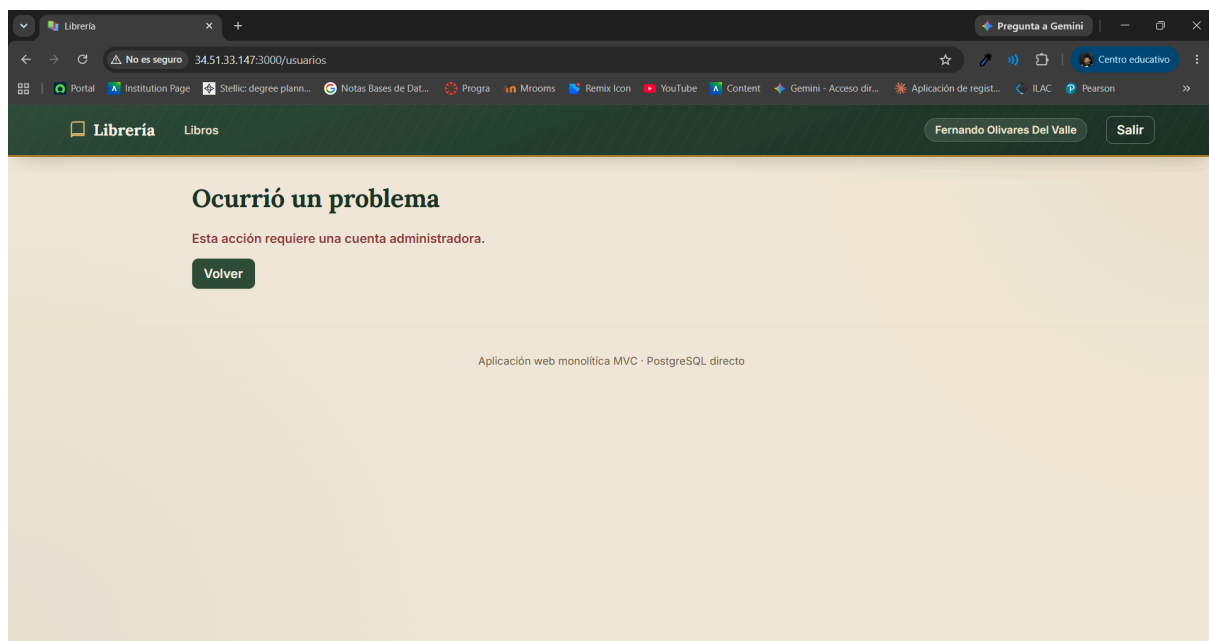
4. Validación server-side de todos los campos

- **Amenaza:** Un atacante puede omitir la validación del navegador (deshabilitando JavaScript o enviando peticiones directas con curl/Postman) y mandar datos inválidos o maliciosos directamente al servidor.
- **Control aplicado:** Además de la validación HTML (required, min, pattern), la base de datos aplica CHECK en precio/stock/año y UNIQUE/FK en las relaciones — de forma que aunque el servidor reciba un dato inválido, PostgreSQL lo rechaza antes de persistirlo, y el manejador de errores central (app.js) traduce esos rechazos en mensajes controlados.
- **Evidencia:** Se envió por curl un INSERT directo con stock negativo — PostgreSQL lo rechazó con el error de CHECK correspondiente (ver docs/DB INTEGRITY.md, pruebas negativas del punto 8).

```
fodv004@maquina-01:/opt/udem/WebMonolitico
library=> INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9999999999999', 'Libro stock malo', 2020, 100.00, -5, 1);
ERROR: new row for relation "libros" violates check constraint "libros_stock_check"
DETAIL: Failing row contains (9999999999999, Libro stock malo, 2020, 100.00, -5, 1, 2026-08-30 16:08:19.496979).
library=>
```

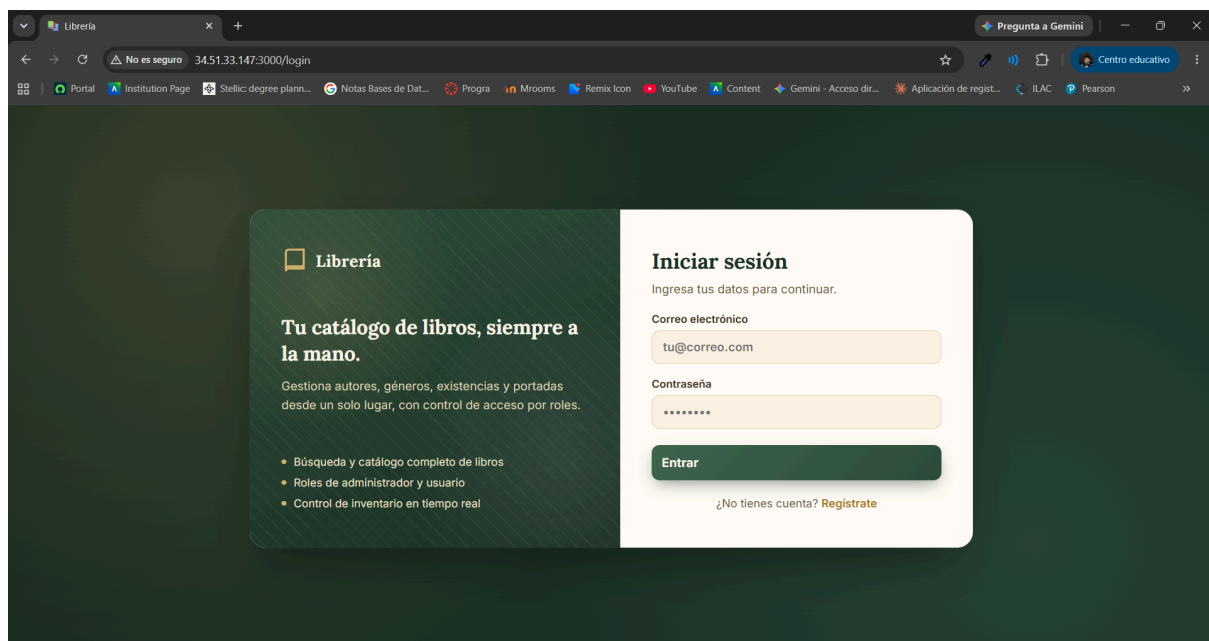
5. Autorización por rol en cada ruta administrativa

- **Amenaza:** Un usuario autenticado pero sin privilegios de Administrador accede a funciones de gestión (crear/editar/eliminar libros, usuarios, catálogos) que no le corresponden.
- **Control aplicado:** El middleware `auth.requireAdmin` (`src/middleware/auth.js`) protege todas las rutas de administración (`/usuarios`, `/catalogos/:type`, creación/edición/borrado de libros) verificando `req.session.user.es_admin` antes de continuar.
- **Evidencia:** Se inició sesión con una cuenta de usuario regular (`usuario1@libreria.com`) y se intentó acceder a `/usuarios` — el servidor respondió con 403 y un mensaje de acceso denegado, sin ejecutar ninguna operación.



6. Manejo seguro de sesiones y cierre de sesión

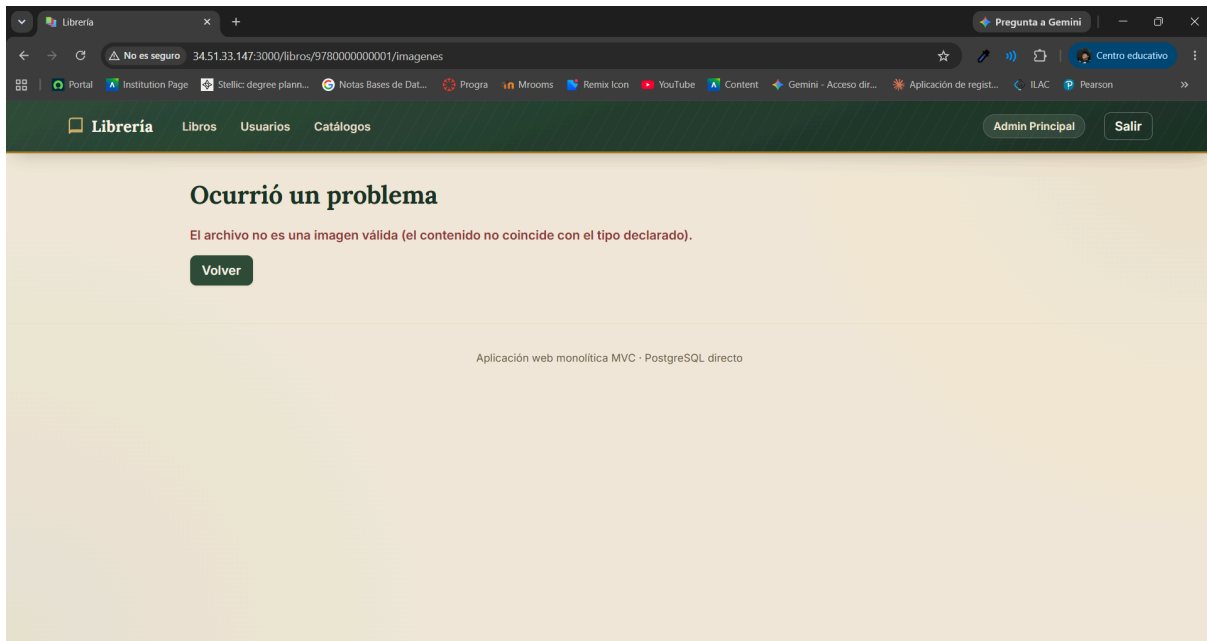
- **Amenaza:** Una sesión mal configurada permite robo de cookie (XSS) o que la sesión persista indefinidamente tras cerrar sesión.
- **Control aplicado:** Express-session se configura con cookie: { httpOnly: true, sameSite: 'lax' } (la cookie no es accesible desde JavaScript del navegador, mitigando robo por XSS) y logout destruye la sesión completa (req.session.destroy) en vez de solo borrar datos parciales.
- **Evidencia:** Tras hacer login y luego POST /logout, una petición subsecuente a una ruta protegida con la misma cookie ya no reconoce al usuario como autenticado.



7. Validación de archivos subidos (extensión, MIME, tamaño, nombre)

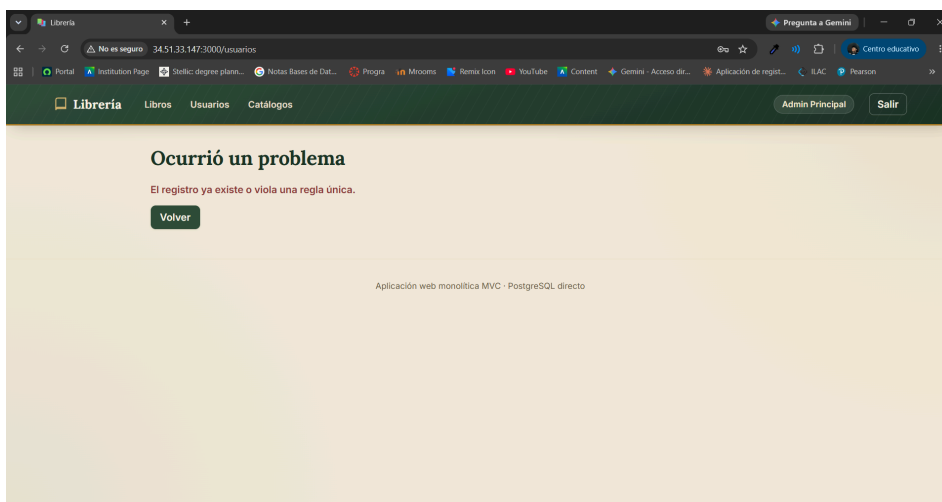
- **Amenaza:** Un atacante sube un archivo ejecutable disfrazado de imagen (por ejemplo, un script con extensión .jpg falsa) para intentar ejecutar código en el servidor, o sube archivos excesivamente grandes para agotar espacio en disco (denegación de servicio).
- **Control aplicado:** Multer valida extensión (.jpg, .jpeg, .png, .webp) y tipo MIME real del archivo simultáneamente, límite de tamaño de 5MB (limits.fileSize), y genera el nombre del archivo en el servidor (timestamp + número aleatorio + extensión) — nunca usa el nombre que envía el usuario.
- **Evidencia:** Se subió un archivo .txt renombrado a .jpg → rechazado con "Formato de imagen no permitido" (el MIME real no coincidía). Se subió un archivo de 6MB → rechazado con "La imagen supera el tamaño máximo permitido (5MB)". Se subió

una imagen JPG válida → se guardó en disco con un nombre generado (1787950956765-665034899.jpg), no con el nombre original.



8. Mensajes de error controlados (sin stack traces ni SQL al usuario)

- **Amenaza:** Exponer el mensaje de error real de PostgreSQL o el stack trace de Node.js revela detalles internos (nombres de tablas, columnas, rutas del servidor) que facilitan un ataque.
- **Control aplicado:** El middleware de errores central en app.js intercepta los errores y los traduce a mensajes genéricos y seguros según el código de error de Postgres (23505 → "El registro ya existe...", 23503 → "No se puede eliminar porque está relacionado...", LIMIT_FILE_SIZE → mensaje de tamaño); el detalle técnico completo solo se imprime en el log del servidor (console.error), nunca en la respuesta HTTP.
- **Evidencia (probada):** Al intentar crear un segundo Administrador, la respuesta HTML mostró "El registro ya existe o viola una regla única." — sin mencionar el nombre de la restricción (un_solo_admin), el nombre de la tabla, ni el stack trace.



9. Principio de mínimo privilegio para el usuario de PostgreSQL

- **Amenaza:** Si la aplicación se conecta con un superusuario de PostgreSQL, cualquier vulnerabilidad en el código (por ejemplo, una inyección SQL no detectada) podría usarse para leer o modificar cualquier base de datos del servidor, crear/eliminar roles, etc.
- **Control aplicado:** La aplicación se conecta exclusivamente con `library_user`, un rol creado sin privilegios de superusuario, con permisos limitados a la base de datos `library_db` (ver `db/00_create_database.sql`).
- **Evidencia:** `db/00_create_database.sql` crea el rol con `CREATE ROLE library_user LOGIN PASSWORD ...` (sin `SUPERUSER`) y otorga privilegios solo sobre `library_db`, no sobre el servidor completo.

```
fodv004@maquina-01:/opt/udem/WebMonolitico
[fodv004@maquina-01 WebMonolitico]$ git ls-files | grep -i env
.env.example
[fodv004@maquina-01 WebMonolitico]$ sudo -u postgres psql
Password for user postgres:
psql (16.14)
Type "help" for help.

postgres=# \du
                                List of roles
Role name | Attributes
-----+-----
library_user |
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS

postgres=#
```

10. No publicar contraseñas, `.env`, claves SSH ni tokens en `ubiquitous.udem.edu`

- **Amenaza:** Publicar credenciales reales en la página de evidencias del ejercicio expone la base de datos (y potencialmente el servidor) a cualquier persona con acceso a la URL pública.
- **Control aplicado:** Antes de comprimir/publicar el proyecto, se excluyen `.env`, `node_modules/`, y cualquier archivo con secretos; solo se publica `.env.example` con nombres de variables y valores de ejemplo (no reales). Los documentos de comandos (`GCP_COMMANDS.md`) y de integridad (`DB_INTEGRITY.md`) usan `*****` o `CAMBIA_ESTA_PASSWORD` en lugar de contraseñas reales.

- **Evidencia:** Revisión manual de docs/GCP_COMMANDS.md y db/00_create_database.sql confirmando que ninguna contraseña real aparece en texto plano.

```
postgres=# exit
[fodv004@maquina-01 WebMonolitico]$ grep -i "password\|contraseña" docs/GCP_COMMANDS.md db/00_create_database.sql
grep: docs/GCP_COMMANDS.md: No such file or directory
db/00_create_database.sql:-- Reemplaza 'CAMBIA_ESTA_PASSWORD' por la contraseña real antes de ejecutar
db/00_create_database.sql:-- y NUNCA publiques este archivo con la contraseña real dentro.
db/00_create_database.sql:CREATE ROLE library_user LOGIN PASSWORD 'CAMBIA_ESTA_PASSWORD';
[fodv004@maquina-01 WebMonolitico]$
```